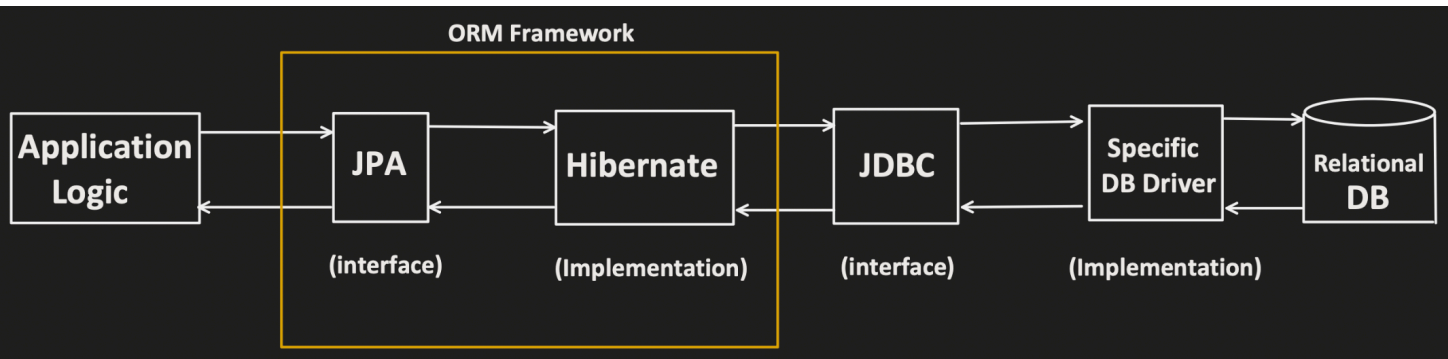
# JPA ARCHITECTURE (PART2)




## ORM (Object-Relational Mapping)

- Act as a bridge between Java Object and Database tables.
- Unlike JDBC, where we have to work with SQL, with this, we can interact with database using Java Objects.

## Lets first see, 1 happy flow first, before deep diving into JPA

pom.xml

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

application.properties

```
#database connection properties
spring.datasource.url=jdbc:h2:mem:userDB
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

```
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    UserDetailsService userDetailsService;

    @GetMapping(path = "/test-jpa")
    public List<UserDetails> getUser() {
        UserDetails userDetails = new UserDetails( name: "xyx",
                email: "xyz@conceptandcoding.com");
        userDetailsService.saveUser(userDetails);
        return userDetailsService.getAllUsers();
    }
}
```

```
@Service
public class UserDetailsService {

    @Autowired
    private UserDetailsRepository userDetailsRepository;

    public void saveUser(UserDetails user) {
        userDetailsRepository.save(user);
    }

    public List<UserDetails> getAllUsers() {
        return userDetailsRepository.findAll();
    }
}
```

```
@Repository
public interface UserDetailsRepository extends
        JpaRepository<UserDetails, Long> {
}
```

```
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // Constructors
    public UserDetails() {
    }

    public UserDetails(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // Getters and setters

}
```